When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

Findings

Of the 18 scenarios taken from the audit, the original system stopped the failure in 4. ERMA-GRC stopped it in all 18. It refused writes without a session, would not let anyone approve their own work, stamped each audit entry with the signed-in user instead of a claimed name, caught an edited entry, and showed a database outage as an error. Beyond the audit, it stopped 3 of 4 failures, including two approvers deciding on one record at the same moment. Stored outcomes were read from the database stand-in, not from what a system claimed. The new design failed twice, and we report both in full. A key risk indicator showed the right status on screen, yet a client calling the server directly could store any status it chose. And anyone holding the audit key could rewrite the trail offline, unnoticed after a restart. The principles also lean on each other: separation of duties is only as sound as the audit trail it reads, and a rule checked only in the browser assures nothing.

Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. The paper offers a taxonomy of failures seen in a working system, seven principles with two testable propositions towards a design theory, and a scenario protocol that others can rerun. Auditors and buyers can use the principles as a checklist. The two failures set the next tasks: anchoring audit trails outside the system, computing every metric on the server, and testing the principles with practitioners.

Keywords: assurance integrity; GRC information systems; design science; enterprise risk management; tamper-evident audit trail; separation of duties

1. Introduction

Risk registers, heatmaps, signed-off assessments and audit trails are now kept mostly in governance, risk and compliance (GRC) information systems. Boards, auditors and regulators read these records as proof that risks are under control (Power, 1997). The proof is worth something only if the screen matches the record underneath.

Our study started with an audit, of code and runtime behaviour, of a GRC application whose documentation called it production-ready. It displayed assurance with nothing behind it. Unrated risks were given ratings worked out from record identifiers or default loss values. Any later save quietly turned an approved assessment back into a draft. The so-called immutable audit trail was stored in the browser, where any user could change it. When the database refused a write, the user was told it had worked. Ordinary design choices had made the system a source of risk instead of a check on it.

We did not find this problem treated in the literature our search reached. Studies of GRC information systems ask why firms adopt them and what value they bring (Papazafeiropoulou & Spanaki, 2016; Racz et al., 2010). Writers on enterprise risk management warn that assurance can slide into ritual (Power, 2009). Security research offers the relevant mechanisms, though in generic form (Clark & Wilson, 1987; Saltzer & Schroeder, 1975). None of the work we found links an application's design failures to corrupted assurance evidence, or tests design principles on a real system.

We therefore ask which design failures in a GRC information system compromise the integrity of its assurance evidence (RQ1); which theory-grounded design principles address them (RQ2); and to what extent an instantiation of these principles prevents the failures compared with the original system (RQ3). Our design science answer is a failure taxonomy, seven principles and a new artifact, ERMA-GRC, tested in time-stamped scenarios. The rest of the paper moves from prior work to method, results, implications and limitations.

2. Literature Review

2.1 Conceptual Background

Enterprise risk management (ERM) is how an organisation as a whole identifies, assesses, treats and monitors its risks. *Governance, risk and compliance (GRC) information systems* bring this work together and hold its records (Racz et al., 2010). From those records, boards, auditors and regulators draw *assurance*, meaning confidence that risks are known and handled. By *assurance integrity* we mean that each assurance item a GRC system shows, whether a rating, an approval, an exposure figure or an audit entry, is either a recorded action that someone can be held to or a plain statement that no record exists. It belongs to the system's design, not to the risks it manages.

2.2 Review of Previous Studies

Organisational research shows that ERM is enacted rather than adopted. Arena et al. (2010) trace how risk rationalities, experts and technologies combine differently across firms; Mikes (2009) identifies distinct "calculative cultures"; Mikes and Kaplan (2015) argue that a single ERM template is itself a risk; and Meidell and Kaarbøe (2017) show that a risk function's tools shape its influence. Power (1997) warns that verification rituals can yield auditability rather than assurance, and Power (2009) calls ERM at its weakest a "risk management of nothing". Technology therefore helps produce risk, so its design merits study.

GRC information systems research has defined the field (Racz et al., 2010), identified implementation success factors (Papazafeiropoulou & Spanaki, 2016), and described an adoption lifecycle and value drivers (Spanaki & Papazafeiropoulou, 2016). It asks how these systems are adopted and what value they deliver, not whether their evidence can be trusted.

The artifact's mechanisms exist in generic form: well-formed transactions, integrity verification and separation of duty (Clark & Wilson, 1987); fail-safe defaults and complete mediation (Saltzer & Schroeder, 1975); separation of duties in workflows (Botha & Eloff, 2001); and tamper-evident logs, using hash chains with evolving keys (Schneier & Kelsey, 1999) or history trees kept honest by external auditors (Crosby & Wallach, 2009). Continuous assurance monitors business-process controls through analytics (Alles et al., 2006; Vasarhelyi et al., 2004), focusing on monitored processes rather than faults in the monitoring software. Table 1 summarises the seven closest studies; five were assessed from abstracts only.

Table 1. Summary of Selected Literature

Author(s)	Context	Methodology	Key Findings	Gap Identified
Racz et al. (2010)	Integrated GRC	Literature analysis, definition	A frame of reference for GRC research	Not its focus: artifact design and evaluation
Papazafeiropoulou & Spanaki (2016)	GRC IS implementation	Literature review and expert interviews	Implementation characteristics and success factors	Integrity of the evidence the systems produce
Spanaki & Papazafeiropoulou (2016)	GRC IS adoption	Qualitative study	Adoption lifecycle; integration, optimisation and information as value drivers	Not its focus: trustworthiness of that information
Arena et al. (2010)	ERM in organisations	Comparative case study	ERM varies with its rationalities, experts and technologies	Not its focus: how a technology's design distorts risk information
Alles et al. (2006)	Continuous control monitoring	Pilot implementation (Siemens)	Business-process controls can be monitored continuously	Not its focus: design faults in the monitoring software

Author(s)	Context	Methodology	Key Findings	Gap Identified
Botha & Eloff (2001)	Workflow access control	Conceptual model	Static and dynamic separation-of-duties enforcement	Not its focus: approval semantics of risk assessments
Crosby & Wallach (2009)	Secure logging	Data structure and analysis	Efficient tamper-evident logs, provided auditors hold commitments	Business workflows and other failure types

2.3 Research Gap

Within the web-indexed literature our search reached, with every DOI verified, no study examines how a GRC application's design can itself corrupt the assurance evidence it presents: by fabricating values, overwriting decisions, accepting unauthenticated authority or reporting failures as success. GRC research stops at adoption and value, while mechanism research does not address the evidence that risk and control self-assessment produces: ratings, approvals, appetite exposure and indicator status. The gap is therefore not a missing mechanism but missing design knowledge that maps application failures to corrupted evidence and tests principles against a real system.

2.4 Research Framework

The study links observed failure modes (RQ1), theory-grounded principles (RQ2), their mechanisms, and scenario outcomes testing whether the failures are prevented (RQ3). These contribute components of a design theory (Gregor & Jones, 2007): purpose and scope, constructs, principles of form and function, and testable propositions (Section 5). Each principle pairs an aim with a mechanism and a kernel-theory rationale (Gregor et al., 2020).

3. Methodology

3.1 Research Design

Our approach is design science (Hevner et al., 2004), following the six steps of Peffers et al. (2007): (1) problem identification, the audit (RQ1); (2) objectives; (3) design of seven principles, instantiated in ERMA-GRC (RQ2); (4) demonstration on Oracle Autonomous Database; (5) evaluation (RQ3); and (6) communication. The evaluation is one artificial, summative episode of the Technical Risk and Efficacy strategy of the Framework for Evaluation in Design Science (FEDS; Venable et al., 2016).

3.2 Data Collection

Evidence came from the audit and a scenario experiment, its protocol time-stamped in the project repository (not a public registry) before execution. The original system (baseline) and ERMA-GRC ran against one database stand-in with identical rows (Figure 1), one recorded run per scenario, keeping full evidence.

3.3 Measures and Instruments

Eighteen of the 23 scenarios operationalise the failure modes, four test the principles beyond them, and one was expected to fail. Runs were coded PREVENTED, OCCURRED or not

applicable against criteria registered in advance, reading stored state from the stand-in and display outcomes from screen text.

3.4 Data Analysis

We count PREVENTED scenarios per failure mode; deterministic outcomes need no inferential statistics. All deviations are reported in a replication package available on request: the test scripts were piloted after registration; open points (e.g., an approver as editor in S2.2) were fixed before each run; the rule coding G4 by its stricter path was not registered; and S5.1 was re-run after a script fault.

Use of generative AI. Claude (Claude Opus 5.5, Anthropic) was used under the authors' direction to assist the audit, implement much of ERMA-GRC, draft the protocol, write and run the test scripts, propose outcome codings, produce the figures, support the literature search, and draft text. The authors set the questions, approved each stage including the protocol before execution, verified sources and results, revised the text and take full responsibility. No AI tool is an author.

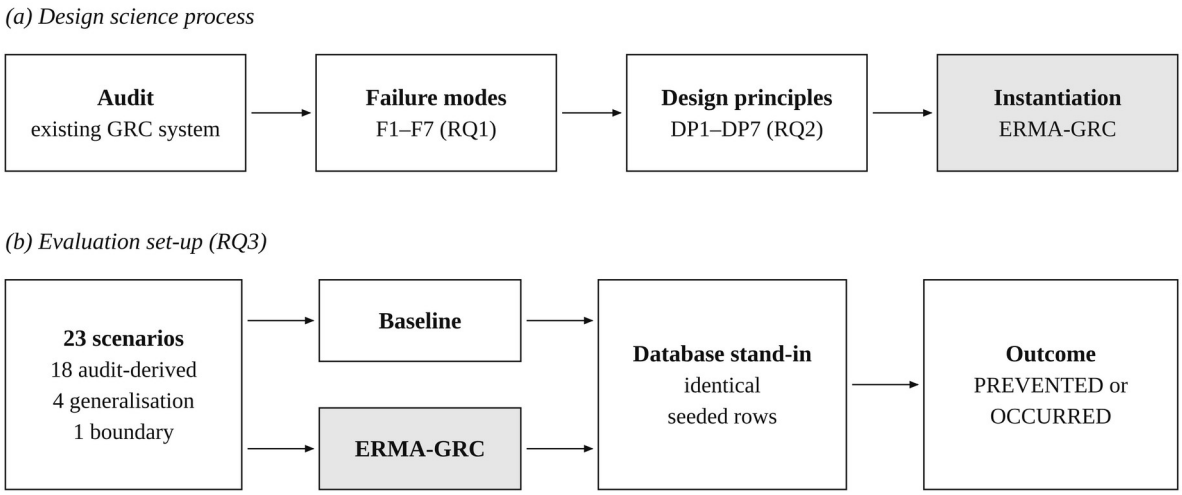


Figure 1. Design science process and evaluation set-up: the audit yields F1–F7, which inform DP1–DP7, instantiated in ERMA-GRC; both systems run the 23 scenarios against one database stand-in.

4. Results and Discussion

4.1 Results

RQ1 and RQ2. Table 2 maps each failure mode to its principle, kernel theory and mechanism in ERMA-GRC.

Table 2. Failure Modes, Design Principles, Kernel Theories and Mechanisms

Failure mode (RQ1)	Design principle (RQ2)	Kernel theory	Mechanism in ERMA-GRC
F1 Fabricated assurance values	DP1 Evidence provenance	Clark & Wilson (1987); Power (1997)	No derived ratings; "Not rated" shown

Failure mode (RQ1)	Design principle (RQ2)	Kernel theory	Mechanism in ERMA-GRC
F2 Decisions overwritten by defaults	DP2 Decision preservation	Clark & Wilson (1987); Saltzer & Schroeder (1975)	Defaults only at creation; approvals only via authorised transitions
F3 Client-side or absent enforcement	DP3 Server-enforced, identity-bound separation of duties	Saltzer & Schroeder (1975); Botha & Eloff (2001)	Server session; deny by default; maker ≠ checker; conflicts refused; revocation
F4 Mutable, forgeable evidence trail	DP4 Tamper-evident, actor-bound trail	Schneier & Kelsey (1999); Crosby & Wallach (2009)	Keyed hash chain (static key); session-bound actor; visible verification
F5 Silent failure reported as success	DP5 Fail-visible operation	Saltzer & Schroeder (1975)	Errors returned; no fallback store
F6 Fragmented system of record	DP6 Single system of record	Clark & Wilson (1987)	Records in database; accounts and audit chain on server
F7 Destructive actions on fabricated records	DP1 Evidence provenance	as for F1	Empty states, no placeholder rows
F1 (metrics)	DP7 Computed, not declared, metrics	Vasarhelyi et al. (2004); Cox (2008)	Status computed from thresholds, in the client

RQ3. ERMA-GRC prevented all 18 audit-derived failures; the baseline prevented 4 (Figure 2). The baseline stored DRAFT after an approver set APPROVED, accepted writes without a session or with a self-declared administrator header, recorded an audit entry under a claimed "CEO" identity, reported success while the database was unreachable, and invented two scored risks, each with a delete button, for a cycle that had none. ERMA-GRC prevented three of the four generalisation failures: it kept one of two simultaneous approval decisions, rejected a write from a deactivated user's still-valid session, and rejected a path-traversal identifier before any database request. The baseline's 0/4 there largely repeats its F2 and F3 defects and is not independent evidence. For balance, when an editor without approval rights changed an approved risk (S2.2 variant), ERMA-GRC returned it to review by design, which the literal criterion codes as OCCURRED; the simultaneous-decision race on a record's status (G1 variant) was prevented.

Where the baseline passed. The baseline displayed a stored statement and single loss expectancy (SLE) faithfully (S1.2), showed a missing approval status as "Draft", as ERMA-GRC does, rather than Approved (S1.3), held no invented entries in a fresh audit trail (S1.4), and attributed an audit entry to the self-asserted acting persona (S4.3). S1.3 and S1.4 contradict the audit report; for S1.4, the seeding code exists but never runs on a fresh installation.

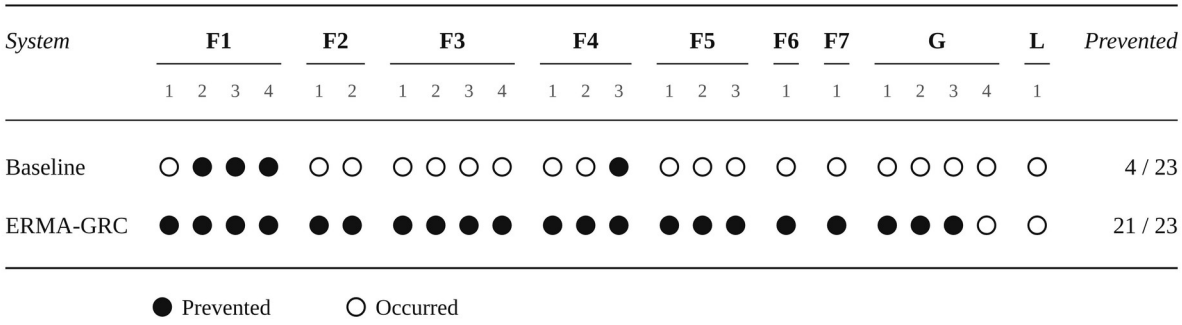


Figure 2. Outcome of every scenario for each system, grouped by failure mode (F1–F7; see Table 2), generalisation scenarios (G) and the boundary scenario (L). Within each group, columns are its scenarios in order (e.g., column 3 under F1 is S1.3). Filled circle: PREVENTED; open circle: OCCURRED.

Where ERMA-GRC failed. In G4, a key risk indicator (KRI) reading received its status by rule in the interface, but the API stored a status declared by the client, as the baseline's does. In L1, an insider holding the audit key re-chained an edited log offline, and verification passed after a restart. Table 3 contrasts two baseline failures with ERMA-GRC on the same stored data.

Table 3. What Each System Showed for Identical Records

Scenario	Database content	Baseline displayed	ERMA-GRC displayed
S1.1 Unrated risk	No likelihood, impact or SLE	Rating 1/5 × 4/5 (score 4, Moderate); SLE $40,000	Inherent and residual "Not rated"; SLE "—"
S7.1 Cycle with no risks	A cycle holding no risks	Two invented risks (scores 16 and 12), each with a Delete button	"No risks in this cycle yet"

4.2 Discussion

The failures are assurance failures. The baseline produced scores, approvals and audit entries disconnected from its stored state: a software-made "risk management of nothing" (Power, 2009). Established mechanisms counter this: fail-safe defaults and complete mediation (Saltzer & Schroeder, 1975), separation of duties (Botha & Eloff, 2001), and well-formed transactions with integrity verification (Clark & Wilson, 1987). DP1, DP2 and DP7 specialise these integrity ideas to assurance evidence: they ask whether a displayed rating, decision or metric is evidence at all.

The principles depend on each other. ERMA-GRC reads a record's creator, and hence who may not approve it, from its audit trail; in L1, rewriting one creation entry changed who was barred from approving that record. DP3 therefore rests on DP4, and DP4 on keys held outside the application. Unlike Schneier and Kelsey's (1999) evolving keys, ERMA-GRC's key is static, and without external commitments its holder can rewrite history (Crosby & Wallach, 2009).

Enforcement must sit at the server. G4 shows that a rule checked only in the client is a convention, not a control. Integrity is necessary but not sufficient: faithful records of a poorly calibrated risk matrix are still poor risk information (Cox, 2008).

5. Implications

Theoretical implications. The study recasts assurance integrity as a design property of GRC systems and contributes components of a nascent design theory (Gregor & Jones, 2007): purpose and scope, constructs, seven principles and two testable propositions. P1: separation of duties is only as strong as the integrity of the evidence trail it reads. P2: an assurance rule enforced only in the client can be defeated by any authenticated client.

Practical implications. Before purchase, buyers and auditors can test whether an unrated risk is shown as unrated, whether an ordinary save can overwrite an approval, whether authority comes from a verified session, whether the audit trail is verifiable and anchored outside the vendor's control, and whether a failed write can ever appear as a success.

Societal implications. Regulators and the public increasingly rely on compliance claims produced by software. Systems that cannot fabricate evidence make those claims more trustworthy and accountability harder to evade.

6. Conclusion, Limitations and Future Research

This study asked how a GRC system's design can corrupt its assurance evidence, and how to prevent it. An audit yielded seven failure modes, addressed by seven theory-grounded principles. ERMA-GRC, which instantiates them, prevented every audit-derived failure; the original system prevented four of eighteen. Its two failures, a metric enforced only in the client and an audit trail its key holder can rewrite, are part of the contribution.

Limitations. The evidence comes from one case. The authors built the artifact and wrote the scenarios with an AI assistant involved in every stage, from audit to outcome coding; time-stamping the protocol and reporting every outcome reduce but do not remove this bias. The evaluation was artificial: both systems ran against a stand-in derived from ERMA-GRC's own test double, so concurrency and outage results depend on its fidelity, and the display scenarios lacked a rated risk as a positive control. The literature search lacked Scopus and Web of Science, and no commercial product was compared.

Future research. Next steps are naturalistic evaluation with practitioners, multiple-case studies across GRC products, externally anchored audit commitments, server-side computation of every assurance metric, and longitudinal study of how integrity-preserving systems change the use of risk information.

References

Alles, M., Brennan, G., Kogan, A., & Vasarhelyi, M. A. (2006). Continuous monitoring of business process controls: A pilot implementation of a continuous auditing system at Siemens. *International Journal of Accounting Information Systems, 7*(2), 137–161. https://doi.org/10.1016/j.accinf.2005.10.004

Arena, M., Arnaboldi, M., & Azzone, G. (2010). The organizational dynamics of enterprise risk management. *Accounting, Organizations and Society, 35*(7), 659–675. https://doi.org/10.1016/j.aos.2010.07.003

Botha, R. A., & Eloff, J. H. P. (2001). Separation of duties for access control enforcement in workflow environments. *IBM Systems Journal, 40*(3), 666–682. https://doi.org/10.1147/sj.403.0666

Clark, D. D., & Wilson, D. R. (1987). A comparison of commercial and military computer security policies. In *Proceedings of the 1987 IEEE Symposium on Security and Privacy* (pp. 184–194). IEEE. https://doi.org/10.1109/SP.1987.10001

Cox, L. A., Jr. (2008). What's wrong with risk matrices? *Risk Analysis, 28*(2), 497–512. https://doi.org/10.1111/j.1539-6924.2008.01030.x

Crosby, S. A., & Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium* (pp. 317–334). USENIX Association. https://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf

Gregor, S., Chandra Kruse, L., & Seidel, S. (2020). Research perspectives: The anatomy of a design principle. *Journal of the Association for Information Systems, 21*(6), 1622–1652. https://doi.org/10.17705/1jais.00649

Gregor, S., & Jones, D. (2007). The anatomy of a design theory. *Journal of the Association for Information Systems, 8*(5), 312–335. https://doi.org/10.17705/1jais.00129

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–106. https://doi.org/10.2307/25148625

Meidell, A., & Kaarbøe, K. (2017). How the enterprise risk management function influences decision-making in the organization – A field study of a large, global oil and gas company. *The British Accounting Review, 49*(1), 39–55. https://doi.org/10.1016/j.bar.2016.10.005

Mikes, A. (2009). Risk management and calculative cultures. *Management Accounting Research, 20*(1), 18–40. https://doi.org/10.1016/j.mar.2008.10.005

Mikes, A., & Kaplan, R. S. (2015). When one size doesn't fit all: Evolving directions in the research and practice of enterprise risk management. *Journal of Applied Corporate Finance, 27*(1), 37–40. https://doi.org/10.1111/jacf.12102

Papazafeiropoulou, A., & Spanaki, K. (2016). Understanding governance, risk and compliance information systems (GRC IS): The experts view. *Information Systems Frontiers, 18*(6), 1251–1263. https://doi.org/10.1007/s10796-015-9572-3

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Power, M. (1997). *The audit society: Rituals of verification*. Oxford University Press.

Power, M. (2009). The risk management of nothing. *Accounting, Organizations and Society, 34*(6–7), 849–855. https://doi.org/10.1016/j.aos.2009.06.001

Racz, N., Weippl, E., & Seufert, A. (2010). A frame of reference for research of integrated governance, risk and compliance (GRC). In B. De Decker & I. Schaumüller-Bichl (Eds.), *Communications and multimedia security* (Lecture Notes in Computer Science, Vol. 6109, pp. 106–117). Springer. https://doi.org/10.1007/978-3-642-13241-4_11

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE, 63*(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

Schneier, B., & Kelsey, J. (1999). Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security, 2*(2), 159–176. https://doi.org/10.1145/317087.317089

Spanaki, K., & Papazafeiropoulou, A. (2016). The implementation of governance, risk, and compliance IS: Adoption lifecycle and enterprise value. *Information Systems Management, 33*(4), 302–315. https://doi.org/10.1080/10580530.2016.1220214

Vasarhelyi, M. A., Alles, M. G., & Kogan, A. (2004). Principles of analytic monitoring for continuous assurance. *Journal of Emerging Technologies in Accounting, 1*(1), 1–21. https://doi.org/10.2308/jeta.2004.1.1.1

Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems, 25*(1), 77–89. https://doi.org/10.1057/ejis.2014.36